企业项目研发流程 | 26 年最新零基础到精通一条龙（万人收藏⭐）

> *编程导航学习网站：学编程、做项目、拿 Offer！*
>
> *企业高频面试题库：开始刷题，面试遇原题！*
>
> *精选简历模板大全：1 分钟搞定简历！*
>
> *AI 资源导航网站：获取最新 AI 黑科技！*
>
> *1 对 1 模拟面试：随时随地提升面试能力*

开篇介绍

很多未工作过的小伙伴都很好奇：企业中做项目是怎样的流程？尤其是大厂那些百万用户的项目，和自己学编程时做项目到底有什么区别呢？

实话说，区别可大了！

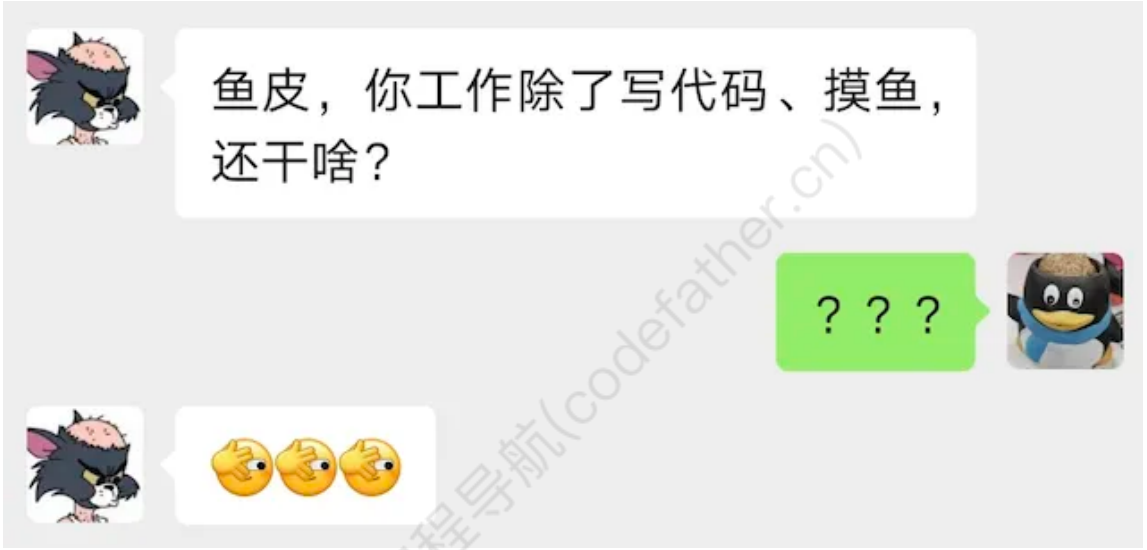
自己开发项目那是单打独斗，自己掌握命运，不会拖垮队友；但企业中开发项目是开团打本，大家是一根绳上的蚂蚱，每个人都会影响整个项目。



鱼皮自己也在几家公司实习过，不得不说，大厂和其他公司的研发流程也有很大的区别。因此，对于大多数同学，如果没有在大厂工作过，对很多研发环节可能都是一无所知的。

所以这篇文章就来揭秘一下大厂的项目研发流程，帮大家开拓思路。正好之前有同志质疑我的日常工作就只有写代码和摸鱼？这篇文章就作为回击，让他明白，在大厂做项目，可不止写代码那么简单！

编程导航(codefather.cn)

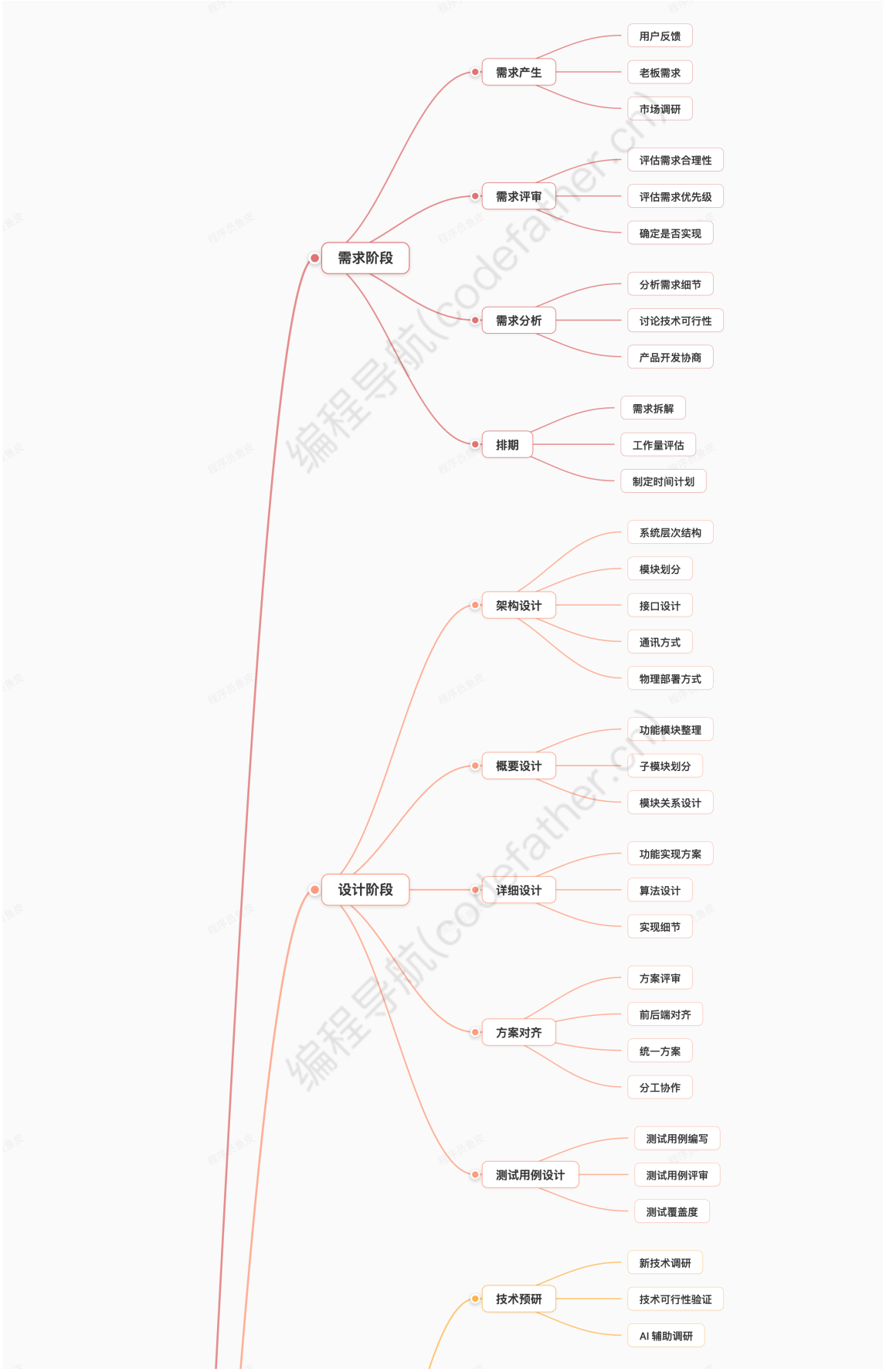


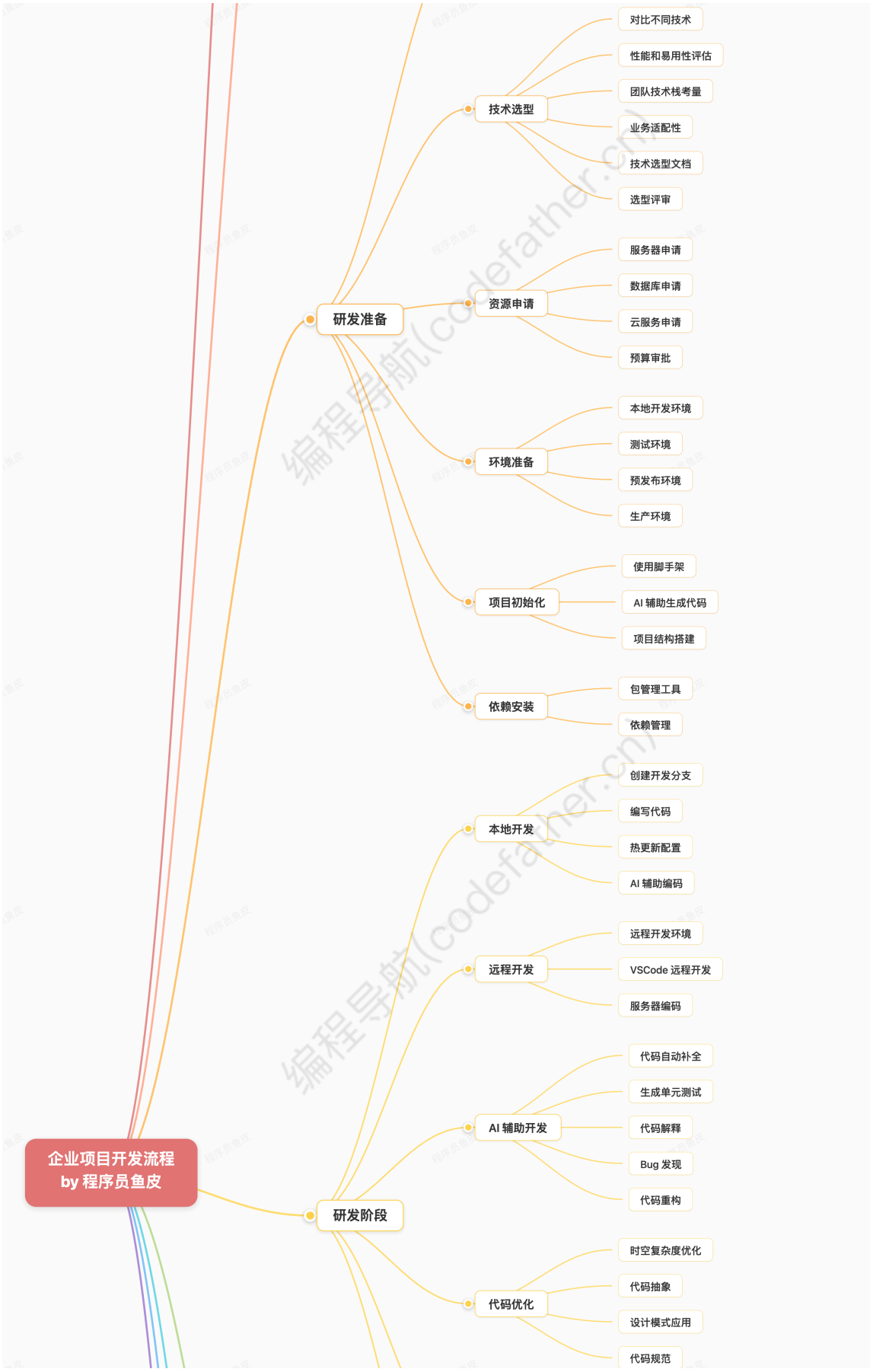
如果你想更直观地了解大厂项目开发流程，强烈推荐观看鱼皮的视频：一线大厂项目是怎么做出来的，结合视频和本文一起学习，效果更好。

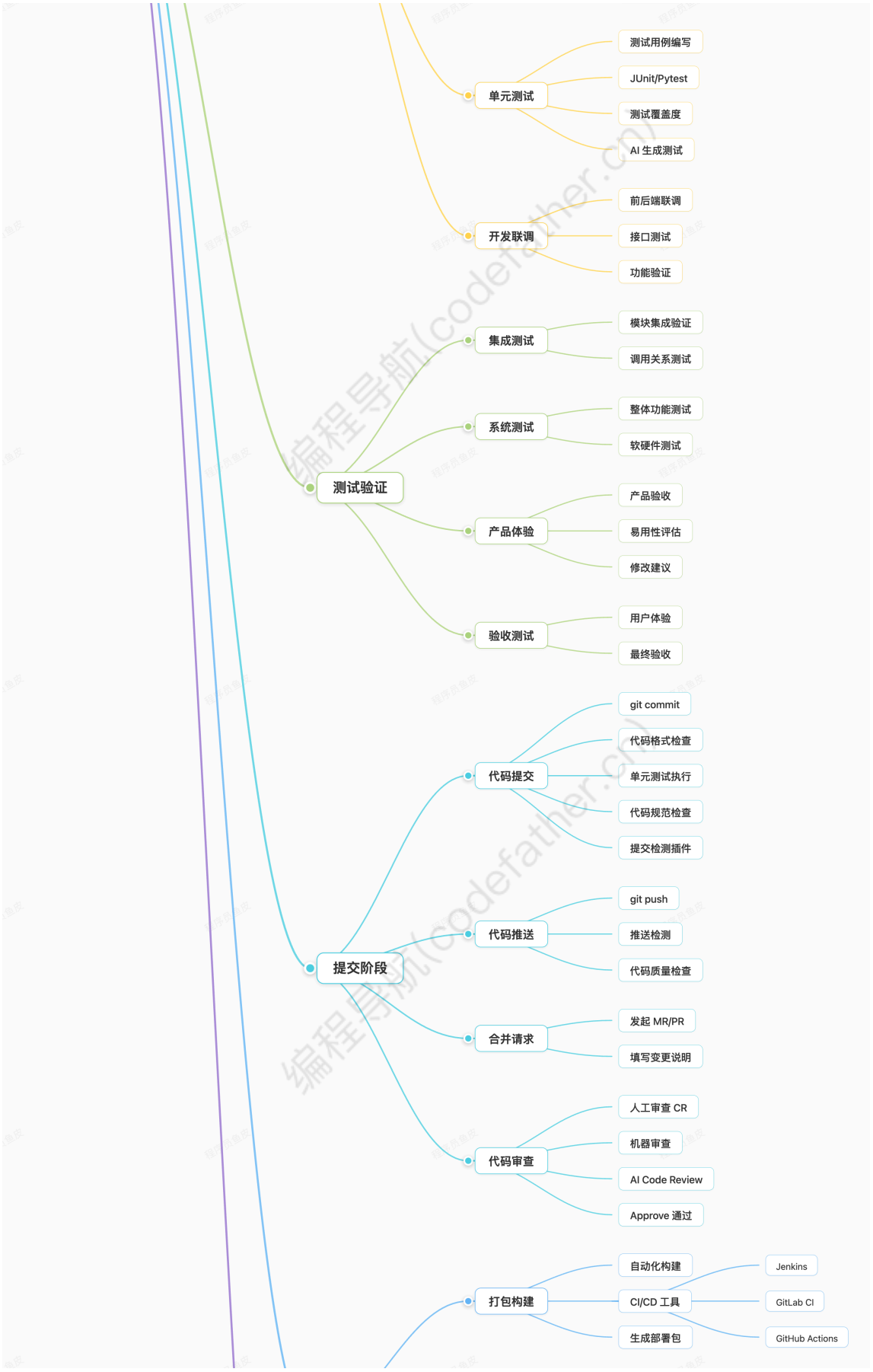
研发流程全景图

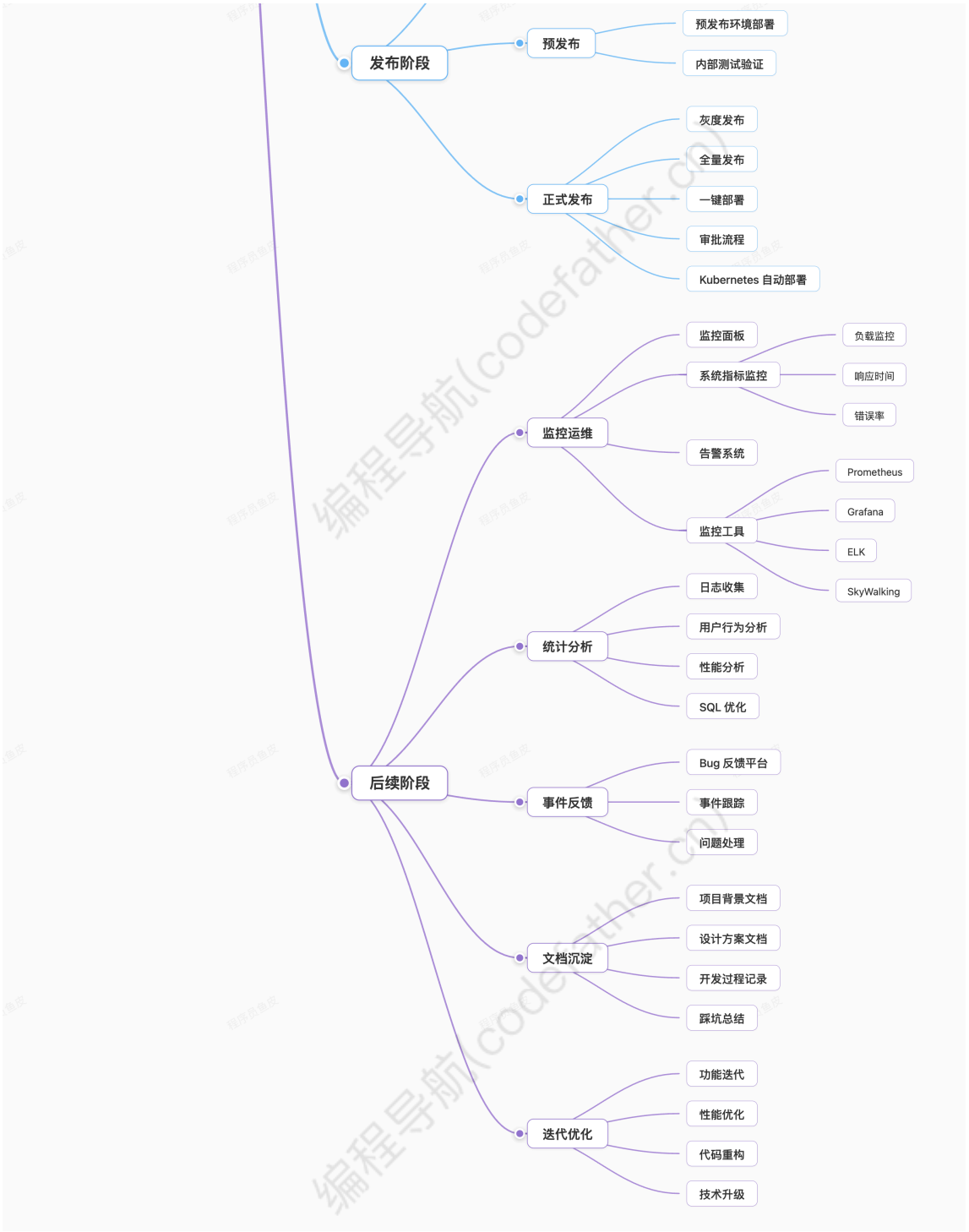
为了规范团队、保证项目的进展，大厂研发流程一般还是比较复杂的。可以分为很多个阶段，用一张思维导图来概括：

编程导航(codefather.cn)


需求阶段
需求产生
用户反馈
老板需求
市场调研
需求评审
评估需求合理性
评估需求优先级
确定是否实现
需求分析
分析需求细节
讨论技术可行性
产品开发协商
排期
需求拆解
工作量评估
制定时间计划
设计阶段
架构设计
系统层次结构
模块划分
接口设计
通讯方式
物理部署方式
概要设计
功能模块整理
子模块划分
模块关系设计
详细设计
功能实现方案
算法设计
实现细节
方案对齐
方案评审
前后端对齐
统一方案
分工协作
测试用例设计
测试用例编写
测试用例评审
测试覆盖度
技术预研
新技术调研
技术可行性验证
AI 辅助调研


企业项目开发流程
by 程序员鱼皮
研发准备
技术选型
对比不同技术
性能和易用性评估
团队技术栈考量
业务适配性
技术选型文档
选型评审
资源申请
服务器申请
数据库申请
云服务申请
预算审批
环境准备
本地开发环境
测试环境
预发布环境
生产环境
项目初始化
使用脚手架
AI 辅助生成代码
项目结构搭建
依赖安装
包管理工具
依赖管理
研发阶段
本地开发
创建开发分支
编写代码
热更新配置
AI 辅助编码
远程开发
远程开发环境
VSCode 远程开发
服务器编码
AI 辅助开发
代码自动补全
生成单元测试
代码解释
Bug 发现
代码重构
代码优化
时空复杂度优化
代码抽象
设计模式应用
代码规范


测试验证
单元测试
测试用例编写
JUnit/Pytest
测试覆盖度
AI 生成测试
开发联调
前后端联调
接口测试
功能验证
集成测试
模块集成验证
调用关系测试
系统测试
整体功能测试
软硬件测试
产品体验
产品验收
易用性评估
修改建议
验收测试
用户体验
最终验收
提交阶段
代码提交
git commit
代码格式检查
单元测试执行
代码规范检查
提交检测插件
代码推送
git push
推送检测
代码质量检查
合并请求
发起 MR/PR
填写变更说明
代码审查
人工审查 CR
机器审查
AI Code Review
Approve 通过
打包构建
自动化构建
CI/CD 工具
生成部署包
Jenkins
GitLab CI
GitHub Actions



需要注意的是，以上阶段并不是完全按从上到下的顺序执行，阶段间可能存在交叉，比如 **技术选型** 其实在 **设计阶段** 就应该考虑。

正式工作一年多，我也是经历过多次项目的完整研发流程的。下面就以我的视角，带大家快速过一遍~

（为了内容更有趣，以下故事有虚构成分）

需求阶段

今天是周一，鱼皮像往常一样骑着他的小电动车来到公司，殊不知，等待他的是一场噩梦的开始。

需求产生

上午十点，产品妹子找到鱼皮，告诉他：咱们的系统上线后，用户表示很多功能并不好用，需要大改。

老板也找到鱼皮，告诉他：我今天打开页面竟然加载了十几秒，咱们这个系统的性能太烂了吧！

鱼皮心想：呕豁，完蛋！估计得做个新的项目了，又要开会了。

编程导航(codefather.cn)



果然，没过多久，屏幕上弹出了一条 "欢迎加入会议" 的邀请。

需求评审

第二天上午，老板、产品、测试、几位开发大哥和鱼皮一起来到会议室，具体讨论昨天提到的那些需求 **是否合理、要不要做**？

产品妹子打开文档，说到：这一期呢，我们要做这几个需求，下面我来详细讲一下，大家一起评估下有没有问题。

需求分析

接下来，产品妹子正在对着屏幕侃侃而谈、疯狂输出时，旁边的开发大哥坐不住了。

开发大哥：这个需求不合理啊！

产品：为啥不合理？用户就是有这个需求啊！

开发大哥：我知道，实现不了啊！

于是开始了经典的产品开发撕逼大战。。。



而鱼皮正躲在角落冷静分析 **这个需求怎么做**，过了一会儿，提出了一种改动低、实现快的解决方案，平息了这场战争。

在 AI 时代，我们还可以用 AI 工具（如 ChatGPT、Cursor）来辅助分析需求，快速生成多种技术方案进行对比，大大提升需求分析的效率。

编程导航(codefather.cn)

排期

确定需求合理、可实现之后，产品妹子问到：那这个需求啥时候能上线呀？

开发大哥：我这周忙，下周吧。

产品：用户可能比较着急，这周就要呢！

开发大哥：我知道，做不完啊！

于是开始了经典的产品开发撕逼大战。。。



鱼皮：要不我们把这个需求拆解为功能 A 和功能 B，这周我先把功能 A 做了，功能 B 排到下周二测试，下周四上线？

就这样，我们一个个安排了需求的计划完成日期。

设计阶段

终于开完会了，看了下时间，都该下班了！

唉，需求讨论完了，产品的工作是完成了一些，可鱼皮的工作才刚刚开始。

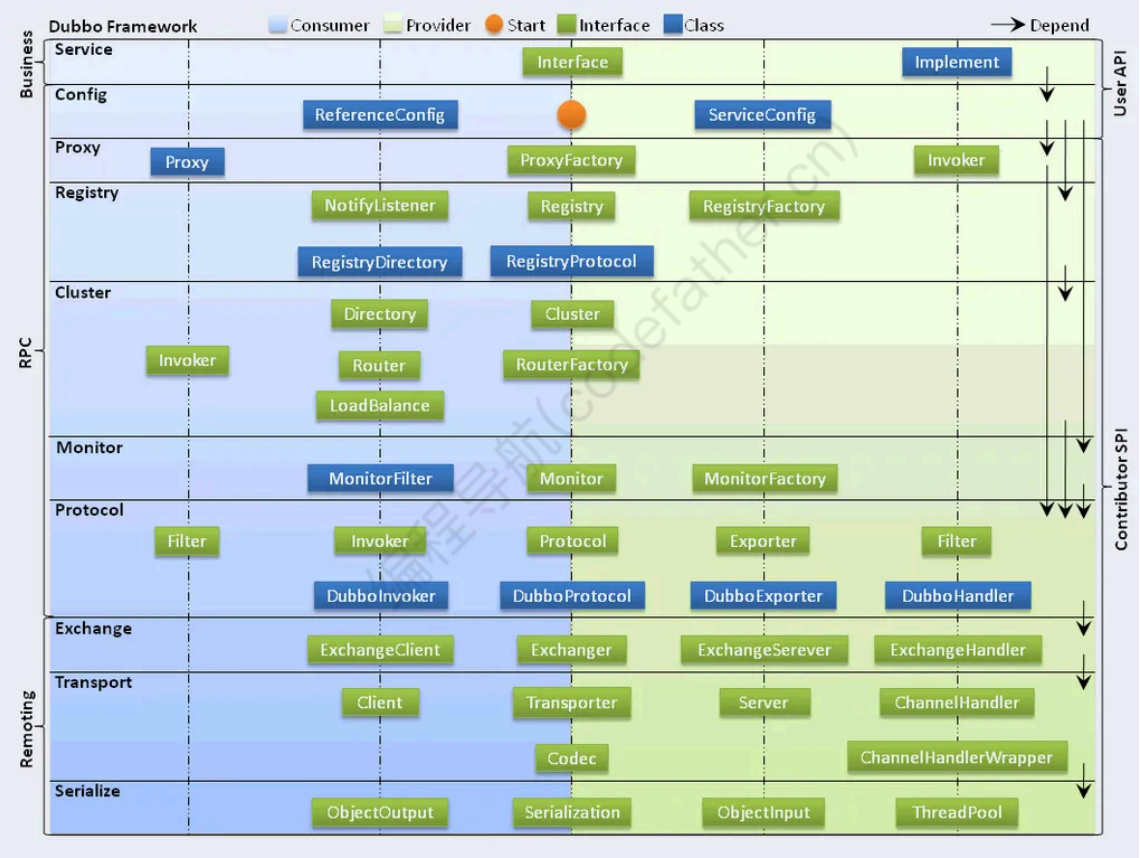
急着开始写代码么？

不，想好怎么写代码比写代码更重要。

架构设计

鱼皮打开写文档软件和画图软件，开始梳理整个系统，从整体到局部，依次设计出系统的层次结构、各层间交互的接口和通讯方式、每层之间包含哪些重要模块、模块选择何种物理部署方式等。

编程导航(codefather.cn)



现在有了 AI 工具的加持，我们可以用 AI 辅助生成架构图、分析架构方案的优缺点，甚至让 AI 帮我们做技术调研，大大提升设计效率。建议观看鱼皮分享的 AI 画图保姆级指南和技巧分享视频。

概要设计

写完架构设计后，鱼皮开始对着产品妹子写的 PRD（产品需求文档），分析需求，然后依然是从整体到局部，先整理出系统需要的功能模块，再分析每个功能模块内有哪些子模块。

和抽象的架构设计相比，概要设计和需求的关系更紧密，是对架构设计的细化。

打个比方大家就明白了，你要盖一栋楼，架构设计就是从整体来考虑，总共有几层、每层管道怎么接、每层有几户、地基怎么打等；而概要设计就是考虑每户套件的内部怎么划分，哪里是客厅、哪里是卫生间。

> *很多情况下，概要设计和架构设计可能会在一个文档中进行，划分并不明确。*

详细设计

想好系统有哪些功能后，鱼皮就开始具体分析每个功能如何实现，用到哪些算法、需要注重哪些细节等。

方案对齐

写好设计文档后，下次会议上，鱼皮和其他的开发同学（前端、后端等）一起针对自己设计的方案展开讨论，最终产生一个统一的方案，然后大家分工去做就好了。

测试用例设计

为了保证系统功能的正常稳定，测试同学（或者叫 QA）是非常重要的。关于软件测试的详细学习，可以查看 软件测试学习路线。测试不是像我们自己做项目一样对着网页点几下就 ok 了。

在大公司中，为了保证测试的覆盖度、提高测试效率，一般是要设计测试用例的，比如：用户点击 "登录"，未传任何数据，期望结果是警告用户输入用户名和密码。



测试用例设计完后，需要其他同学一起来评审把关，而不是只交给测试同学。因为一个人很容易忽略掉很多测试细节，最好让更熟悉代码的开发同学一起帮忙补充。

鱼皮自己也写了几个测试可能会遗漏的用例，和测试同学一起进行了确认，尽量让问题暴露在测试阶段而不是线上。

研发准备

写了快一周的设计文档，终于准备开始动手搭建项目了。但在此之前，还有一些准备工作要进行。

技术预研

如今技术发展太快，新技术层出不穷，所以鱼皮首先对项目中需要或可能需要用到的技术进行了调研。

在 AI 时代，技术调研变得更加高效。我们可以用 AI 工具快速了解新技术、对比不同方案，甚至让 AI 帮我们生成技术选型报告。

技术选型

通过调研，鱼皮得到了几个可以满足需求的技术，但他开始纠结：这么多技术，我该用哪一个呢？是用 SSM 框架还是 Spring Boot 呢？用 Guava 包还是 Apache Commons 呢？

鱼皮又打开了写文档软件，开始对比不同技术的优劣，头疼啊，技术选型要考量的因素太多了，比如：

- 单从技术考虑：性能、易用性、稳定性、主流程度和生态、文档详细度
- 结合团队：团队成员对技术的熟悉度、掌控度（有无精通该技术的人）
- 结合业务：是否适应业务的量级（单机 or 微服务）、是否适应业务（读多、写多 or 分析多）

对于关键的项目，鱼皮自己还不敢完全确定选型，因此在写好自己的选型文档后，和同事和 Leader 一起讨论，才最终确认。

资源申请

确认好技术后，就要申请资源。比如鱼皮用到了 MySQL 数据库，但是这个 MySQL 从哪儿来呢？

以前的话，鱼皮都是去买一台云服务器，自己搭建 MySQL。但是在企业中，一般是有集中管理和分配资源的平台的，直接到平台填写预算、等领导审批、然后等着下发资源就好了。千万不能私自用自己的或买外部的服务器来部署项目，不安全！

鱼皮这次直接申请到了 2 万多一年的云数据库，真的是爽死了。



环境准备

申请好数据库等资源后，鱼皮按照申请机器的版本搭建了一模一样的本地开发环境和测试环境，后面就可以直接连接了。

项目初始化

环境准备妥当后，由于是新项目，鱼皮要搞一个最小可运行的初始化项目 Demo，使用 **脚手架** 自动生成代码，而不是从零开始一个个新建文件、手敲重复代码。

现在还有更智能的方式：使用 AI 工具（如 Cursor、GitHub Copilot）来辅助初始化项目、生成模板代码，大大节省时间。

依赖安装

生成了项目代码后，鱼皮使用包管理工具（前端 yarn/npm、Java Maven / Gradle 等）自动安装依赖，然后项目 Demo 就可以运行啦！

研发阶段

前期准备完成后，这才到了程序员朋友们最熟悉的写代码环节，也是鱼皮最爱的环节。



因为之前设计方案时需要保持冷静、仔细思考，没法边听歌儿边做；而方案设计好后，已经明确了该怎么做，写代码实现就很简单了，顶多是遇到一些坑，上网搜索去解决就好了。

本地开发

开发时，一般鱼皮会先在本地写代码，通过配置热更新工具，实现代码更新时自动重新编译打包，而不用手动重启项目，大大提高了开发效率。

对了，企业开发都会使用版本控制系统的，比如 Git，开发前记得先创建一个自己的分支，在这个分支上开发。

远程开发

现在还有一种比较流行的远程开发方式，就是可以像编辑本地文件一样编辑远程文件，直接修改服务器上的代码。一般我们每位研发同学是有自己的开发机的，通过远程开发就省去了反复部署调试的麻烦，提高效率。一般用 VSCode 等开发工具，安装远程开发插件就可以实现了。鱼皮之前分享过一个简单的 VSCode 远程开发实操视频，可以看一下。

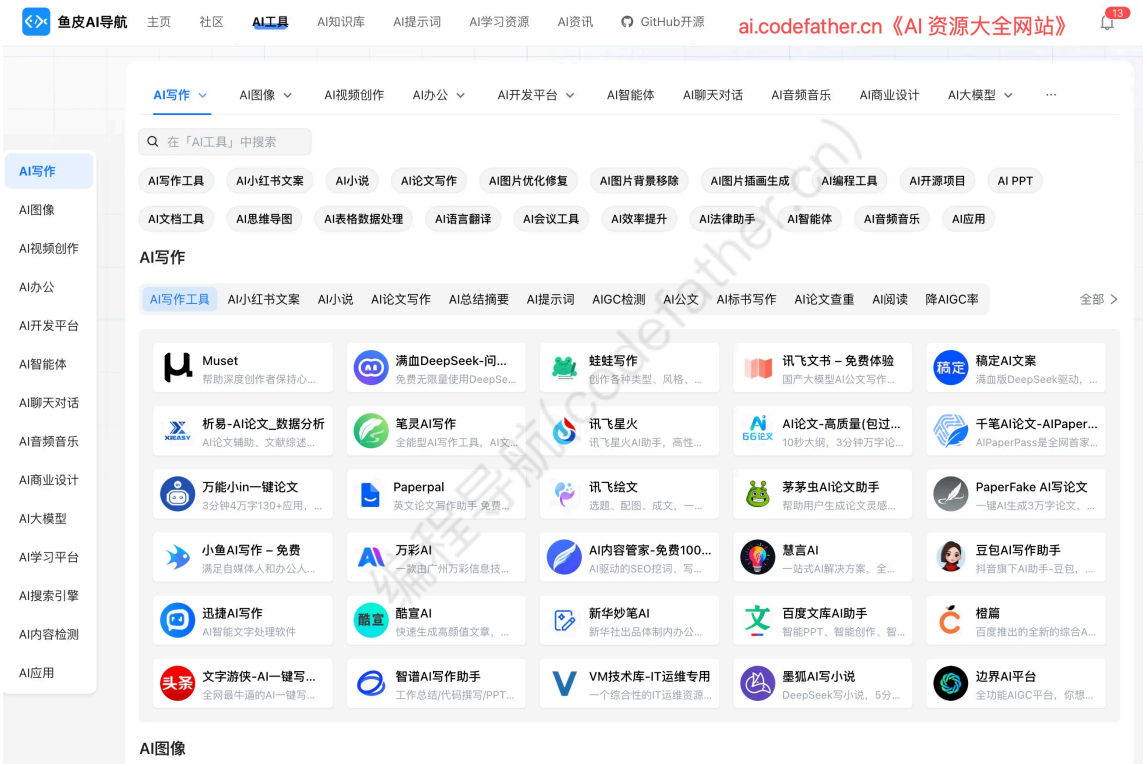
AI 辅助开发

在 2025 年，AI 辅助开发已经成为主流。鱼皮现在写代码时经常使用 Cursor、GitHub Copilot 等 AI 工具，它们能够：

- 自动补全代码，提升编码效率
- 生成单元测试，保证代码质量
- 解释复杂代码，帮助理解
- 发现潜在 Bug，提前预警
- 重构优化代码，提升可维护性

推荐大家使用 AI 资源大全 中的工具，提升开发效率。

编程导航(codefather.cn)



代码优化

鱼皮在写代码的时候，始终保持主动优化代码的好习惯，注重代码的时空复杂度；并且当重复代码多了，会想办法抽象成函数或者使用设计模式。之前专门写文章分享过我的编程习惯：[我写代码时的小倔强]()。

单元测试

注意！不要听到测试就以为是测试同学的工作，开发同学也同样需要编写小粒度的测试来为自己的代码负责。

鱼皮一般会为每个数据库读写函数和业务逻辑函数编写单元测试，像 Java 的话一般用 JUnit 等工具，还可以用 Jacoco 生成测试覆盖度报告。每次修改关键代码后，都要执行一遍单元测试，防止意外错误。

编程导航(codefather.cn)

JaCoCo

Element	Missed Instructions	Cov.	Missed Branches	Cov.
org.jacoco.examples		58%		64%
org.jacoco.core		97%		93%
org.jacoco.agent.rt		77%		84%
jacoco-maven-plugin		90%		80%
org.jacoco.cli		97%		100%
org.jacoco.report		99%		99%
org.jacoco.ant		98%		99%
org.jacoco.agent		86%		75%
Total	1,357 of 27,492	95%	150 of 2,179	93%

现在还可以用 AI 工具自动生成单元测试代码，省去很多重复劳动。

开发联调

鱼皮终于写好了后端代码，也自测完成了，下面就是把写好的代码打包构建，然后把可执行项目包发布到测试服务器上，和前端同学一起联调，让他请求我的接口，验证系统的功能是否可用。

测试验证

鱼皮和前端联调完毕后，告知了测试和产品同学。

测试验证是企业中至关重要的环节，甚至可以说是最后一道防线。测试的目的是找 Bug，尽量发现系统中的问题，把它们扼杀在测试阶段。

在企业中，测试验证又有很多类型。

集成测试

集成测试比单元测试粒度更大，是把多个模块或代码单元放在一起，验证模块之间的集成和调用关系。

因为单个函数的执行可能是正常的，但把多个函数组合在一起顺序调用，可能就会出现问题。

打个比方，我们有个吃面包系统：

功能 A：小鱼吃一个面包

功能 B：小皮吃一个面包

每次只有一个面包，独立执行功能 A 和 B 都是允许的。但如果两个一起执行，后执行的那个功能就会报错。

编程导航(codefather.cn)



系统测试

系统测试比集成测试的粒度更大，测试对象是整个系统，不仅包括软件，还可能覆盖对硬件的测试。

产品体验

除了测试同学要验证系统可用性，产品妹子也要体验下功能是否符合预期、是否易用。大多数情况下，产品会在体验时提出修改建议，开发可能还要再去做一些修改。

验收测试

测试和产品妹子终于表示没有问题啦，那就到了最后一步，把整个产品或功能给最终的用户来体验。老板/用户说没问题，才是真的没问题！

提交阶段

系统没问题之后，鱼皮就可以把代码发布到远程仓库了，一般使用 Git 和 SVN 等版本控制系统。

代码提交

鱼皮首先在本地触发代码提交（git commit），为保证规范，在大项目中一般会使用提交检测插件（如 Husky、pre-commit hook），防止你把错误的代码进行了提交。

现代的代码提交还会自动触发：

- 代码格式检查（Prettier、ESLint）
- 单元测试执行
- 代码规范检查（CheckStyle、SonarLint）

代码推送

下一步就是把本地的提交推送到远程的同名分支。一般大厂会有推送检测工具，检测代码的错误、圈复杂度、代码规范等，和提交检测一样，防止你把错误或不规范的代码进行了推送。

合并请求

代码分支推送到远程之后，鱼皮发起了一个分支合并请求（MR 或 PR），希望把该分支的代码合并到主干分支（没问题的代码）。



代码审查

并不是发起了合并请求就能直接合并，还要通过代码审查，即 CR（Code Review）。

审查又分为两种方式：人审和机审。

相信不少同学都知道人审，一般是由你的上级和其他项目负责人来阅读和评论你的代码，觉得没问题就 Approve（通过），否则打回去修改。

那机审是个啥呢？其实就是机器自动检测你的代码是否符合规范，是否能够成功自动化构建等，一般是由项目负责人配置的，可以帮助发现一些人工难以发现的问题。

刚接触新项目的时候，鱼皮经常被机审折磨得苦不堪言，经常被提示一些莫名其妙的代码问题，比如加号要换行，文件行末要加空行等。但后来注意编码习惯后，就很自然地适应了，的确不错。

现在还有 AI Code Review 工具，能够自动发现代码中的问题、提出优化建议，进一步提升代码质量。

发布阶段

代码审查通过后，鱼皮的项目代码就可以发布上线啦。



打包构建

传统上线方式是开发人员到正式服务器上拉取代码，然后安装依赖，再通过工具把代码打包构建，得到部署包，通过 Nginx、Tomcat、Docker 等技术运行。

但这样效率很低，有很多重复工作。所以大厂一般是用自动化构建的，像 Jenkins、GitLab CI、GitHub Actions 等 CI/CD 工具。代码合并到主分支后，由机器自动把代码打包构建为最终的部署包。

预发布

为了防止上线出问题，一般我们会先在预发布环境部署项目，再观察一下是否能够正常运行。预发布环境的配置和生产环境完全一致，但只有内部人员能访问。

正式发布

编程导航(codefather.cn)

预发布测试正常后，鱼皮终于等到了上线的这一刻。大项目一般都会部署在多台机器上，所以不可能一台台登录机器去发布部署包。

一般公司会提供可视化发布平台，点选需要发布的机器（一般先灰度，选一小部分机器，再全量发布），点击一键发布，等项目管理员审批通过之后，就交给机器自动部署吧！

在云原生时代，很多公司使用 Kubernetes 进行容器编排和自动部署，发布流程变得更加自动化和标准化。

后续

鱼皮曾天真地以为项目上线之后，就可以高枕无忧了。但后来发现，项目上线之后，同样需要保持警觉。虽然已经测试过，但仍然时不时会出现个预期之外的小 Bug，还是很考验心态的。



来看看上线之后，鱼皮做了哪些事呢？

监控运维

鱼皮会定期查看项目的监控面板，观察项目的运行情况、机器的负载、接口的响应时间、错误率等关键指标。

现代的监控系统非常强大，可以实时监控系统的各项指标，一旦发现异常就会立即告警。常用的监控工具有：

- Prometheus + Grafana：开源监控方案
- ELK（Elasticsearch + Logstash + Kibana）：日志收集和分析
- SkyWalking：分布式追踪系统
- 云厂商的监控服务（阿里云监控、腾讯云监控等）

鱼皮把自己学到的东西分享了出来，如果想要了解大厂是怎么监控项目的，可以看看 Prometheus + Grafana 可观测性实战保姆级教程视频。

统计分析

鱼皮在代码中添加了一些日志，可以利用 ELK 等日志收集可视化平台对这些日志进行分析，从而感知到用户的行为，进一步优化业务和系统。

比如我会统计用户执行 SQL 查询的耗时，对重复率高的慢 SQL 进行针对性地优化。

事件反馈

有的时候，用户自己都不能清楚地描述 Bug，而且历史 Bug 也不方便找到。所以公司内部一般会有事件反馈平台，产品等内部同学在接收到 Bug 时，会在该平台发布一个 Bug 事件，详细描述 Bug 出现的时间、状况、详情等，便于我们开发集中分析和处理问题。



文档沉淀

每次上线了新功能和项目，鱼皮都会通过写文档来记录项目的背景、设计方案、开发过程和一些坑点，便于后续其他同学了解项目，这是非常重要的！利人利己。

好的文档能够：

- 帮助新人快速了解项目
- 记录技术决策和演进历程
- 避免重复踩坑
- 方便后续维护和迭代

迭代优化

最后，一个需求的结束往往只是另一个需求的开始。像鱼皮最近在跟进的项目，一期做完做二期，二期还没做完三期就来了；还要抽出时间去优化以前的代码，这日子遥遥无期，没盼头啊！

研发流程对比

为了让大家更清楚地理解企业项目开发和个人项目开发的区别，鱼皮整理了一个对比表格：

环节	个人项目	企业项目
需求	自己想做什么就做什么	需求评审、需求分析、排期
设计	想到哪写到哪	架构设计、概要设计、详细设计、方案评审
开发	一个人写代码	多人协作、分支管理、代码规范
测试	自己点点看	单元测试、集成测试、系统测试、测试用例
部署	随便部署	预发布、灰度发布、全量发布

上线后	不管了	监控、日志分析、Bug 修复、迭代优化

可以看到，企业项目开发的流程要复杂得多，但这些流程都是为了保证项目的质量和稳定性。

学习建议

如何学习企业项目开发流程?

1）通过项目实战学习：最好的学习方式是参加真实的企业项目。如果还是学生或者刚入职，建议多观察团队的开发流程，主动参加各个环节。

2）跟着教程做完整项目：如果没有企业项目经验，可以跟着 鱼皮的实战项目教程 学习。编程导航的项目教程 都是按照企业开发流程来设计的，包含需求分析、架构设计、开发、测试、部署等完整环节。

3）学习 Git 和 CI/CD：版本控制和自动化部署是企业开发的基础，要重点学习。建议学习 Git 学习路线 和 CI/CD 持续集成学习路线。

4）了解敏捷开发：现代互联网公司大多采用敏捷开发，要了解 Scrum、Sprint、每日站会等概念。

推荐的学习路线

总结一下，如果你想系统学习企业项目开发，建议按以下顺序学习相关技术：

1. Git 和 GitHub 学习路线：掌握版本控制
2. 软件工程学习路线：理解开发规范和设计模式
3. DevOps 工程师学习路线：了解 DevOps 实践
4. CI/CD 学习路线：掌握自动化部署
5. 跟着 鱼皮的项目教程 做完整项目：体验企业级项目开发流程

推荐资源

视频教程

- ⭐ 鱼皮的大厂项目开发流程视频：一线大厂项目是怎么做出来的

项目实战

- ⭐ 编程导航 - 鱼皮原创项目教程系列：企业级项目开发完整流程
- 项目学习建议：如何学习项目

工具平台

- Git：版本控制系统
- GitHub / GitLab：代码托管平台
- Jenkins：自动化构建工具
- Jira：项目管理工具
- 禅道：国产项目管理工具

技术博客

- Atlassian Blog：敏捷开发和项目管理
- GitLab Blog：DevOps 和协作流程
- Spotify Engineering：Spotify 团队协作
- Netflix TechBlog：Netflix 研发效能

写在最后

编程导航(codefather.cn)

看完本文后，相信大家对企业项目开发流程有了更清晰的认识。企业开发和个人开发最大的区别在于：企业开发强调流程、规范、协作、质量保证，而个人开发更注重快速实现功能。

如果你想进入大厂工作，建议提前了解企业项目开发流程，学习相关的工具和方法论。跟着鱼皮的项目教程做几个完整的项目，实践企业级项目的开发流程，这会让你在求职和工作中更有竞争力。

推荐阅读鱼皮之前的文章：大厂机密！30 个提升团队研发效能的锦囊，了解更多大厂研发实践。

加油，鱼友们！



程序员必备资源

1）程序员学习交流圈：极客教程、实战项目、求职宝典

2）程序员面试八股文：实习/校招/社招高频考点、企业真题解析

3）程序员写简历神器：专业模板、丰富例句、直通面试

4）AI 知识资源大全：前沿技术、最新 AI 资讯、提示词大全

5）1 对 1 模拟面试：实习/校招/社招面试拿 Offer 必备

编程导航(codefather.cn)